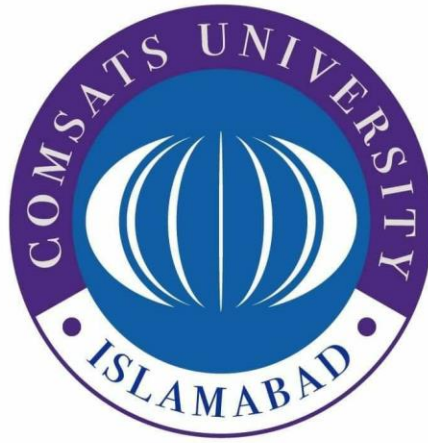




Department of Computer Science,
CUI, Attock Campus.

Program: BSSE

Assignment no:4

Information Security (LAB)

Student Name	Ahsan Rehman Khan
Registration #	FA24-BSE-056

Professor: Ms. Ambreen Gul

Task 1:

Sol:

Input:

```
# GCD -----
def gcd(a, b):
    while b > 0:
        temp = a % b
        a = b
        b = temp
    return a

# Modular Inverse -----
def mod_inverse(e, phi):
    d = 1
    while d < phi:
        if (e * d) % phi == 1:
            return d
        d += 1

# Key Generation -----
p = 11
q = 13
n = p * q
phi = (p - 1) * (q - 1)

e = 7
while gcd(e, phi) != 1:
    e += 1
d = mod_inverse(e, phi)

public_key = (e, n)
private_key = (d, n)
print("Public Key:", public_key)
print("Private Key:", private_key)
```

```

# Encryption -----
def encrypt(message):
    e, n = public_key
    encrypted = []
    for i in range(len(message)):
        char = message[i]
        num = ord(char)
        result = 1
        for j in range(e):
            result = result * num

        cipher = result % n
        encrypted.append(cipher)

    return encrypted

# Decryption -----
def decrypt(cipher):
    d, n = private_key
    decrypted = ""
    for i in range(len(cipher)):
        num = cipher[i]
        result = 1
        for j in range(d):
            result = result * num

        value = result % n
        char = chr(value)
        decrypted = decrypted + char

    return decrypted

```

```

# Testing -----
message = "Ahsan Rehman Khan Niazi"

enc = encrypt(message)
print("Encrypted:", enc)

dec = decrypt(enc)
print("Decrypted:", dec)

```

Output:

```

Public Key: (7, 143)
Private Key: (103, 143)
Encrypted: [65, 91, 80, 59, 33, 98, 69, 62, 91, 21, 59, 33, 98, 114, 91, 59, 33, 98, 78, 118, 59, 34, 118]
Decrypted: Ahsan Rehman Khan Niazi
PS D:\python (VS code)>

```

Task 2:

Sol:

Input:

```
# GCD -----
def gcd(a, b):
    while b > 0:
        temp = a % b
        a = b
        b = temp
    return a

# Modular Inverse -----
def mod_inverse(e, phi):
    d = 1
    while d < phi:
        if (e * d) % phi == 1:
            return d
        d += 1

# Key Generation -----
p = 11
q = 13

n = p * q
phi = (p - 1) * (q - 1)
e = 7
while gcd(e, phi) != 1:
    e += 1

d = mod_inverse(e, phi)
public_key = (e, n)
private_key = (d, n)
```

```

# Simple Hash Function -----
def simple_hash(message):
    h = 0
    for ch in message:
        h = h + ord(ch)
    return h % 100    # small hash value

# Sign (using private key) -----
def sign(message):
    d, n = private_key
    h = simple_hash(message)

    result = 1
    for i in range(d):
        result = result * h
    signature = result % n
    return signature

# Verify (using public key) -----
def verify(message, signature):
    e, n = public_key
    h = simple_hash(message)

    result = 1
    for i in range(e):
        result = result * signature
    check = result % n
    if check == h:
        return True
    else:
        return False

```

```

# Testing -----
message = "AOA"

signature = sign(message)
print("Message:", message)
print("Signature:", signature)

# Verify correct message
print("Verification (original):", verify(message, signature))

# Modify message
fake_message = "AoA"    # small change
print("Verification (modified):", verify(fake_message, signature))

```

Output:

```
t_03.py
Message: AOA
Signature: 113
Verification (original): True
Verification (modified): False
PS D:\python (VS code)> |
```

Task 1 Explanation:

1. Key Generation

There are 2 keys:

- A public key used for encrypting the message
- A private key used for decrypting the message

2. Encryption

The original message is converted into numbers and transformed into a secure form so that it cannot be understood by others

3. Decryption

The encrypted numbers are converted back into the original message using the private key

4. Testing

The system checks whether the message is correctly encrypted and then successfully decrypted back to the original form

Task 2 Explanation:

1. Key Generation

There are 2 keys:

- Public key used for verifying the signature
- Private key used for creating the signature

2. Hash Function

The message is converted into a small numeric value called a hash. Even a small change in the message will change this hash value.

3. Signing

The hash value of the message is encrypted using the private key. This creates a digital signature which proves the message is authentic.

4. Verification

The signature is checked using the public key. The system compares the original hash and the computed value. If both match, the message is valid; otherwise, it is invalid.

5. Testing

First, the original message is signed and verified successfully. Then the message is slightly changed. After modification, verification fails because the hash no longer matches.